

custom variables :-

A **custom variable** is simply a variable you create in your Jenkinsfile.

```
pipeline {
    agent any

    stages {
        stage('Custom Variable') {
            steps {
                script {
                    def appName = "Azhari-App"
                    def version = "1.0"

                    echo "Application: ${appName}"
                    echo "Version: ${version}"
                }
            }
        }
    }
}
```

`def appName = ...` is Groovy code.

The `script {}` block allows us to execute arbitrary Groovy inside a Declarative Pipeline.

Expected output:

```
Application: Azhari-App
Version: 1.0
```

Environment variables :-

Jenkins has an `environment` directive:

```
environment {
    APP_NAME = 'Azhari-App'
    APP_VERSION = '1.0'
}
```

These become **environment variables available to Pipeline steps**. Jenkins supports environment variables globally at the pipeline level or only inside a particular stage.

```

pipeline {
    agent any

    environment {
        APP_NAME = 'Azhari-App'
        APP_VERSION = '1.0'
    }

    stages {
        stage('Environment Variables') {
            steps {
                echo "Application: ${env.APP_NAME}"
                echo "Version: ${env.APP_VERSION}"
            }
        }
    }
}

```

When :-

```

stage('Deploy') {

    when {
        expression {
            env.ENVIRONMENT == 'production'
        }
    }

    steps {
        echo "Deploying to production"
    }
}

```

The `when` condition decides whether the **whole stage executes**.

Jenkins supports built-in conditions such as `environment`, `expression`, `branch`, `anyOf`, `allOf`, and `not`

when { expression } :-

```

when {
    expression {
        env.ENVIRONMENT == 'production'
    }
}

```

```
}  
}
```

think of it:-

ENVIRONMENT

|



Is it "production"?

/

YES NO

| |



Run stage Skip stage

The `expression` contains a **Groovy expression that must evaluate to true/false**

example:-

```
pipeline {  
  agent any  
  
  environment {  
    ENVIRONMENT = 'production'  
    APP_NAME = 'Azhari-App'  
  }  
  
  stages {  
  
    stage('Build') {  
      steps {  
        script {  
          def version = "1.0"  
  
          echo "Building ${env.APP_NAME}"  
          echo "Version = ${version}"  
        }  
      }  
    }  
  
    stage('Deploy') {  
  
      when {  
        expression {  
          env.ENVIRONMENT == 'production'  
        }  
      }  
    }  
  }  
}
```

```

    }
  }
  steps {
    echo "Deploying ${env.APP_NAME}"
    echo "Environment = ${env.ENVIRONMENT}"
  }
}
}
}

```

Expected result:

Build

↓

SUCCESS

Deploy

↓

ENVIRONMENT == production

↓

TRUE

↓

Deploy runs

user input :-

```

pipeline {
  agent any
  stages {
    stage ('Build'){
      steps {
        echo "Building the application"
        echo " Build completed successfully"
      }
    }
    stage ('Deployment approval'){
      input{
        message "Do u want to approve the deployment?"
        ok "Approve the deployment"
        parameters{
          string (
            name: 'Approver_Name',
            defaultValue: 'Avej',
            description: 'Enter your name'
          )
        }
      }
    }
  }
}

```

```

        )
    }
}
steps {
    echo "Deployment Approved by ${Approver_Name}"
    echo "starting deployment"
}
}
stage ('Deploy'){
    steps{
        echo "Deploying the application"
        echo "Deployment completed"
    }
}
}
}

```

input {
 message → What question should Jenkins ask?
 ok → What should the button say?
 submitter → Who can approve?
 parameters → What information should they enter?
}

example:-

```

input {
    message "Approve production deployment?"
    ok "Deploy"
    submitter "admin"
    parameters {
        string(
            name: 'APPROVER_NAME',
            defaultValue: 'Avej',
            description: 'Who is approving?'
        )
    }
}

```

Jenkins supports all four of those configuration options in the Declarative `input` directive.

```

pipeline {
    agent any

```

```

stages {
  stage ('Build'){
    steps{
      echo "Building an application"
      echo "successfully build the application"
    }
  }
  stage ('Deployment Approval'){
    input {
      message "Do you want to deploy"
      ok "continue"
      parameters{
        choice (
          name: "Deploy",
          choices: ["yes","No"],
          description: "enter your name"
        )
      }
    }
    steps {
      echo "Deployment decision received by ${Approver_Name}"
    }
  }
  stage ('Deploy'){
    when{
      expression{
        params.Deploy == "yes"
      }
    }
    steps{
      echo "Deploying the application"
      echo "Deployment finised"
    }
  }
}
}

```

function :-

```

pipeline{

  agent any

  environment{
    myacc_name= "${params.Account_name}"
  }
}

```

```

    }
    parameters{
        choice(
            name: "Account_name",
            choices: ["Dev","Qa"]
        )
    }

    stages{
        stage ('deploy in Dev'){
            when{
                expression{
                    params.Account_name == "dev"
                }
            }
            steps{
                sh "echo Deploying in Dev account"
                getAccinfo(env.my_acc_name)
            }
        }
        stage('Deploy in Qa'){
            when{
                expression{
                    params.Account_name == "Qa"
                }
            }
            steps{
                sh "echo Deploying in Qa account"
                getAccinfo(env.myacc_name)
            }
        }
    }
}

def getAccinfo( String myacc_name){
    if (myacc_name == "Dev"){
        sh "echo Hello from function name getAccinfo , FYI aws account id"
        sh "echo env acc id =1234id"
    }
    else{
        sh "echo Hello from function name getAccinfo , FYI aws account id"
        sh "echo acc id 123qa"
    }
}

```

Docker build-push using pipeline :-

```
pipeline{
    agent any

    stages{

        stage ("checkout"){

            steps{

                checkout scm

            }

        }

        stage('Build Docker image'){

            steps{

                script{

                    def app= docker.build("avejlanjekar45/jenkins-docker-
pipeline:${GIT_COMMIT}")

                    docker.withRegistry(

                        'https://registry.hub.docker.com',

                        'dockerhub-credentials'

                    )

                    {

                        app.push()

                    }

                }

            }

        }

    }
}
```

```
}  
  
}
```

same pipeline using env vars

```
pipeline {  
    agent any  
  
    environment {  
        DOCKER_IMAGE = 'avejlanjekar45/jenkins-docker-pipeline'  
        DOCKER_CREDENTIALS = 'dockerhub-credentials'  
        DOCKER_REGISTRY = 'https://registry.hub.docker.com'  
    }  
  
    stages {  
  
        stage('Checkout') {  
            steps {  
                checkout scm  
            }  
        }  
  
        stage('Build Docker Image') {  
            steps {  
                script {  
  
                    def app = docker.build(  
                        "${DOCKER_IMAGE}:${GIT_COMMIT}"  
                    )  
  
                    docker.withRegistry(  
                        "${DOCKER_REGISTRY}",  
                        "${DOCKER_CREDENTIALS}"  
                    ) {  
                        app.push()  
                        app.push('latest')  
                    }  
                }  
            }  
        }  
    }  
}
```

try-catch :-

In Groovy/Jenkins:

```
try {  
    // code that might fail  
}  
catch (Exception e) {  
    // what to do if it fails  
}
```

try

↓ Run something

↓ Did it fail?

└─ NO → continue

└─ YES → catch ↓ handle error

```
pipeline {  
    agent any  
  
    stages {  
  
        stage('Try Catch Demo') {  
            steps {  
                script {  
  
                    try {  
                        echo 'Starting operation...'  
  
                        sh 'exit 1'  
  
                        echo 'This will NOT execute'  
                    }  
                    catch (Exception e) {  
                        echo 'Something went wrong!'  
                        echo "Error: ${e}"  
                    }  
                }  
            }  
        }  
    }  
}
```

sh 'exit 1' :- `exit 1` means: The shell command failed.

So:

```
echo Starting operation
  ↓
sh 'exit 1'
  ↓
  ✖
  ↓
catch
  ↓
echo Something went wrong!
```

```
pipeline {
    agent any

    environment {
        DOCKER_IMAGE = 'avejlanjekar45/jenkins-docker-pipeline'
        DOCKER_REGISTRY = 'https://registry.hub.docker.com'
        DOCKER_CREDENTIALS = 'dockerhub-credentials'
    }

    stages {

        stage('Checkout') {
            steps {
                checkout scm
            }
        }

        stage('Build Docker Image') {
            steps {
                script {

                    try {

                        def app = docker.build(
                            "${DOCKER_IMAGE}:${GIT_COMMIT}"
                        )

                        docker.withRegistry(
                            "${DOCKER_REGISTRY}",
                            "${DOCKER_CREDENTIALS}"
                        ) {
```

```

        app.push()
        app.push('latest')
    }

    echo 'Docker build and push successful'

} catch (Exception e) {

    echo 'Docker build or push failed'
    echo "Error: ${e}"

}

}

}

}

}

}

```

Shared Library :-

Suppose you have 10 projects:

```

Project A → Jenkinsfile
Project B → Jenkinsfile
Project C → Jenkinsfile
...
Project J → Jenkinsfile

```

If every Jenkinsfile contains the same:

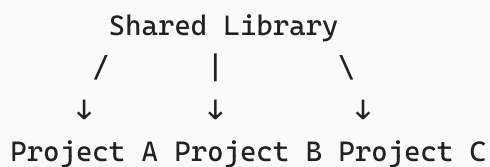
```

docker.build(...)
docker.withRegistry(...)
app.push()

```

you have to maintain the same code in 10 places.

A Shared Library lets us do:



The reusable Jenkins code lives in a **separate Git repository**.

to configure shared library in jenkins :-

jenkins -> Manage jenkins -> system -> Global trusted pipeline

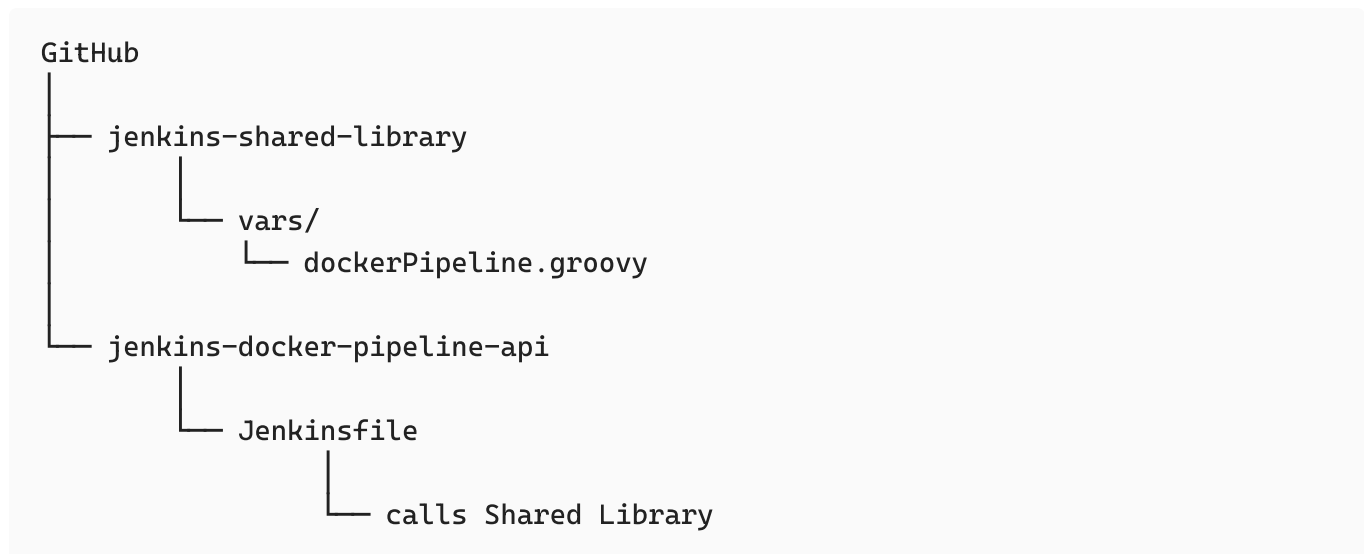
click add , give name to shared library (as we have given 'my-shared-library') , default version = main , Git , add repo , select credentials

Lab:- We'll create two GitHub repositories:

Repository 1 — Shared Library:- jenkins-shared-library

Repository 2 — Application :- jenkins-docker-pipeline

Architecture:



The application Jenkinsfile becomes very small:

```
@Library('my-shared-library') _

dockerPipeline()
```

Instead of putting the entire Docker pipeline inside every project.

Step 1 — Create Shared Library repository :-

Create a **new empty GitHub repository**:

```
jenkins-shared-library
```

```
mkdir jenkins-shared-library
cd jenkins-shared-library
```

Initialize Git:

```
git init
git branch -M main
```

Step 2 — Understand the required structure :-

A Jenkins Shared Library has a special directory structure.

```
jenkins-shared-library/
└─ vars/
```

Create it:

```
mkdir vars
```

so it will be:-

```
jenkins-shared-library
└─ vars
```

Step 3 — Our first shared function :-

Inside `vars` , create:

```
vim vars/dockerPipeline.groovy
```

```
cat dockerPipeline.groovy

def call(String imagename){
    echo "Building docker image ${imagename}"
    def app= docker.build("${imagename}:${GIT_COMMIT}")

    docker.withRegistry(
        'https://registry.hub.docker.com',
        'dockerhub-credentials'
    )
    {
        app.push()
    }
}
```

```
}  
}
```

step 4 — create another directory

it will have:-

```
avej_lanjekar@AvejLanjekar:jenkins-pipeline-shared-library$ ls  
Dockerfile Jenkinsfile index.html
```

inside Jenkinsfile:-

```
avej_lanjekar@AvejLanjekar:jenkins-pipeline-shared-library$ cat Jenkinsfile  
  
@Library('my-shared-library') _  
pipeline{  
    agent any  
    stages{  
        stage('Check Workspace') {  
            steps {  
                sh 'pwd'  
                sh 'ls -la'  
            }  
        }  
        stage('test-shared-library'){  
            steps{  
                dockerPipeline('avejlanjekar45/jenkins-docker-pipeline-using-  
sl')  
            }  
        }  
    }  
}
```

Another Example :-

```
avej_lanjekar@AvejLanjekar:vars$ cat /var/dockerPipeline.groovy  
  
def call(){  
    pipeline{  
        agent any  
  
        environment{  
            Docker_image= 'avejlanjekar45/jenkins-docker-pipeline-using-sl'  
            Registry_url= 'https://registry.hub.docker.com'        }  
    }  
}
```

```

        Credentials='dockerhub-credentials'
    }

    stages{
        stage("checkout"){
            steps{
                checkout scm
            }
        }

        stage("docker build & Push"){
            steps{
                script{
                    def app=docker.build("${Docker_image}:${GIT_COMMIT}")

                    docker.withRegistry(
                        "${Registry_url}",
                        "${Credentials}"
                    )
                    {
                        app.push()
                    }
                }
            }
        }
    }
}
}
}

```

now inside another repo (main app in which we want to use that pipeline) :-

```

avej_lanjekar@AvejLanjekar:jenkins-pipeline-shared-library$ cat Jenkinsfile

@Library('my-shared-library') _
dockerPipeline()

```

Docker Image Promotion :-

Suppose Jenkins builds your application:

Source Code



```
GitHub
  ↓
Jenkins
  ↓
Docker Build
  ↓
Docker Image
```

You might have:

```
myapp:abc123
```

where `abc123` is the Git commit SHA.

Now your application needs to move through environments:

```
Development
  ↓
Testing
  ↓
Staging
  ↓
Production
```

A common mistake is to **rebuild the Docker image at every environment**.

For example:

```
Code
  ↓
Build image
  ↓
Dev

Code
  ↓
Build image again
  ↓
Staging

Code
  ↓
Build image again
```

↓
Production

This means you're potentially deploying **different image artifacts**.

Image promotion solves this

Instead, build the image **once**:

```
Git Commit
  ↓
Jenkins
  ↓
Docker Build
  ↓
myapp:abc123
```

Then promote that **same image** through environments.

```
myapp:abc123
  |
  ▼
  DEV
  |
  tested
  |
  ▼
  STAGING
  |
  approved
  |
  ▼
  PROD
```

The key principle is:

Build once, promote the same artifact.

You don't rebuild the application for production.

Promotion usually means **giving the same image another tag** or copying it to another registry/repository.

For example:

```
myapp:abc123
```

has been tested successfully.

You can tag the same image:

```
myapp:staging
```

and later:

```
myapp:production
```

Docker image promotion :-

```
Build Docker Image in Dev  
Push Dev DH
```

```
Fetch from Dev DH ----  
tag  
push to QA DH
```

```
Fetch from QA DH ----  
tag  
push to DH stage
```

```
Fetch from Stage DH ----  
tag  
Push to DH Prod
```

```
pipeline{

    agent any

    environment{

        DEV_DOCKER_IMAGE= 'avejlanjekar45/dev-docker-image'

        QA_DOCKER_IMAGE= 'avejlanjekar45/qa-docker-image'

        STAGE_DOCKER_IMAGE= 'avejlanjekar45/stage-docker-image'


        REGISTRY_URL= 'https://registry.hub.docker.com'

        DEV_DH_CREDENTIALS= 'dev-dockerhub-credentials'

        QA_DH_CREDENTIALS= 'qa-dockerhub-credentials'

        STAGE_DH_CREDENTIALS= 'stage-dockerhub-credentials'

    }


    stages{

        stage("checkout"){

            steps{

                checkout scm

            }

        }


        stage("Build docker image"){

            steps{
```

```

        script{

            def app= docker.build(

                "${DEV_DOCKER_IMAGE}:${GIT_COMMIT}"

            )


            docker.withRegistry(
"${REGISTRY_URL}", "${DEV_DH_CREDENTIALS}")
            {

                app.push()

            }

        }

    }
}

```

```

stage('promoto dev to qa'){

    steps{

        script{

            docker.withRegistry(

                "${REGISTRY_URL}",

                "${DEV_DH_CREDENTIALS}"

            ){

                docker.image("${DEV_DOCKER_IMAGE}:${GIT_COMMIT}").pull()

            }

```

```

        sh "docker tag ${DEV_DOCKER_IMAGE}:${GIT_COMMIT}
${QA_DOCKER_IMAGE}:${GIT_COMMIT}"

docker.withRegistry("${REGISTRY_URL}", "${QA_DH_CREDENTIALS}") {

docker.image("${QA_DOCKER_IMAGE}:${GIT_COMMIT}").push()

        }

    }

}

stage('promote QA to stage'){

    steps{

        script{

docker.withRegistry("${REGISTRY_URL}", "${QA_DH_CREDENTIALS}") {

docker.image("${QA_DOCKER_IMAGE}:${GIT_COMMIT}").pull()

        }

        sh "docker tag ${QA_DOCKER_IMAGE}:${GIT_COMMIT}
${STAGE_DOCKER_IMAGE}:${GIT_COMMIT}"

docker.withRegistry("${REGISTRY_URL}", "${STAGE_DH_CREDENTIALS}") {

```

```

docker.image("${STAGE_DOCKER_IMAGE}:${GIT_COMMIT}").push()

    }

}

}

}

}

}

```

Above is the image promotion

concept is first we build image from dev to dev DH

then we pull image from dev DH and then tag that image as QA then we push to QA DH

then we pull image from QA and then tag that image to stage then we push to stage DH

```

pipeline{

    agent any

    environment{

        DEV_DOCKER_IMAGE= 'avejlanjekar45/dev-docker-image'

        QA_DOCKER_IMAGE= 'avejlanjekar45/qa-docker-image'

        STAGE_DOCKER_IMAGE= 'avejlanjekar45/stage-docker-image'

        PROD_DOCKER_IMAGE= 'avejlanjekar45/prod-docker-image'

        REGISTRY_URL= 'https://registry.hub.docker.com'

        DEV_DH_CREDENTIALS= 'dev-dockerhub-credentials'

        QA_DH_CREDENTIALS= 'qa-dockerhub-credentials'
    }
}

```

```
    STAGE_DH_CREDENTIALS= 'stage-dockerhub-credentials'

    PROD_DH_CREDENTIALS= 'prod-dockerhub-credentials'
}
```

```
stages{
```

```
    stage("checkout"){
```

```
        steps{
```

```
            checkout scm
```

```
        }
```

```
    }
```

```
    stage("Build docker image"){
```

```
        steps{
```

```
            script{
```

```
                def app= docker.build(
```

```
                    "${DEV_DOCKER_IMAGE}:${GIT_COMMIT}"
```

```
                )
```

```
                docker.withRegistry(
```

```
                    "${REGISTRY_URL}",
```

```
                    "${DEV_DH_CREDENTIALS}"
```

```
                )
```

```
        {  
            app.push()  
        }  
    }  
}  
}
```

```
stage('promoto dev to qa'){
```

```
    steps{
```

```
        script{
```

```
            docker.withRegistry(  
                "${REGISTRY_URL}",  
                "${DEV_DH_CREDENTIALS}"  
            ){
```

```
                docker.image("${DEV_DOCKER_IMAGE}:${GIT_COMMIT}").pull()
```

```
            }
```

```
            sh "docker tag ${DEV_DOCKER_IMAGE}:${GIT_COMMIT}  
${QA_DOCKER_IMAGE}:${GIT_COMMIT}"
```

```
            docker.withRegistry("${REGISTRY_URL}", "${QA_DH_CREDENTIALS}") {
```

```
                docker.image("${QA_DOCKER_IMAGE}:${GIT_COMMIT}").push()
```

```

    }

    }

}

stage('promote QA to stage'){

    steps{

        script{

docker.withRegistry("${REGISTRY_URL}", "${QA_DH_CREDENTIALS}") {

docker.image("${QA_DOCKER_IMAGE}:${GIT_COMMIT}").pull()

        }

        sh "docker tag ${QA_DOCKER_IMAGE}:${GIT_COMMIT}
${STAGE_DOCKER_IMAGE}:${GIT_COMMIT}"

docker.withRegistry("${REGISTRY_URL}", "${STAGE_DH_CREDENTIALS}") {

docker.image("${STAGE_DOCKER_IMAGE}:${GIT_COMMIT}").push()

        }

    }

}

}

```

```

        stage('Promote stage to Prod'){

            steps{

                script{

docker.withRegistry("${REGISTRY_URL}", "${STAGE_DH_CREDENTIALS}") {

docker.image("${STAGE_DOCKER_IMAGE}:${GIT_COMMIT}").pull()

                }

                sh "docker tag ${STAGE_DOCKER_IMAGE}:${GIT_COMMIT}
${PROD_DOCKER_IMAGE}:${GIT_COMMIT}"

docker.withRegistry("${REGISTRY_URL}", "${PROD_DH_CREDENTIALS}") {

docker.image("${PROD_DOCKER_IMAGE}:${GIT_COMMIT}").push()

                }

            }

        }

    }

}

```

Jenkins -Docker - K8s (minikube) :-

```

pipeline{

    agent any

```

```
environment{

    DOCKER_IMAGE='avejlanjekar45/jenkins-docker-k8s'

    REGISTRY_URL='https://registry.hub.docker.com'

    CREDENTIALS= 'dockerhub-credentials'

}


stages{

    stage('checkout'){

        steps{

            checkout scm

        }

    }


    stage('docker build & push'){

        steps{

            script{

                def app=docker.build("${env.DOCKER_IMAGE}:${GIT_COMMIT}")

                docker.withRegistry(

                    "${env.REGISTRY_URL}",

                    "${env.CREDENTIALS}"
```

```

        ) {
            app.push()
        }
    }
}

stage('deploy to k8s'){
    steps{
        withKubeConfig([
            credentialsId: 'minikube-kubeconfig'
        ]){
            sh "kubectl apply -f kube/deployment.yaml"

            sh "kubectl apply -f kube/service.yaml"

            sh "kubectl set image deployment/pipeline-deployment
jenkins-k8s-app=${env.DOCKER_IMAGE}:${env.GIT_COMMIT}"

            sh "kubectl rollout status deployment/pipeline-deployment"
        }
    }
}
}
}

```

first we need to install kubernetes cli

then create credentials for kubeconfig as secret file , upload the secret file of kubeconfig (need

to create kubeconfig file)